

1 NixOS & Home Manager Optimization Report (Final)

1.1 1. Goal & Constraints

The objective is to optimize a multi-host NixOS configuration (t420, t490s) that uses a standalone Home Manager setup.

Constraints:

1. Standalone Home Manager: Must remain separate to allow independent update cycles (System = Stable/Slow, User = Bleeding Edge/Fast).
2. Scalability: The structure must support adding more machines easily, not just the current two.
3. Minimal Redundancy: Code duplication (packages, config) must be eliminated.
4. No "Unified Flake": We will NOT merge the system and home manager flakes.

1.2 2. Architecture Plan

We will adopt a Module-Based Architecture. Instead of having "per-host" configuration files that duplicate 90% of the content, we will create small, reusable modules. Hosts will simply import the modules they need.

1.2.1 2.1 Directory Structure

We will refactor the home-manager/ directory as follows:

```
home-manager/
├── flake.nix          (Keeps tracking nixos-unstable for user apps)
├── modules/           (Reusable Logic)
│   ├── core/
│   │   └── default.nix (Base settings: username, stateVersion, home-manager.enable)
│   ├── shell/         (Shell environment)
│   │   ├── fish.nix
│   │   └── starship.nix
│   ├── desktop/       (GUI Environment)
│   │   ├── hyprland.nix
│   │   ├── waybar.nix
│   │   └── fonts.nix
│   └── packages/       (Package Groups)
│       ├── base.nix    (Browser, File Manager, Media Players - Common)
│       ├── dev.nix     (Coding tools: Helix, Git, Languages)
│       ├── gaming.nix  (Steam, Lutris - Optional)
│       └── creative.nix (Krita, Inkscape - Optional)
└── hosts/             (Machine Definitions)
    ├── t420.nix        (Imports: core, shell, desktop, packages/base)
    └── t490s.nix       (Imports: core, shell, desktop, packages/all)
```

1.2.2 2.2 The "Slowness" Explanation

You rightly observed that downloading 50MB of nixpkgs isn't the main bottleneck; evaluation is. When you run home-manager switch, Nix has to parse thousands of Nix files in nixpkgs to build the dependency graph.

Why your setup is currently slow: You are evaluating the entire nixpkgs tree twice: once for the system (during nixos-rebuild) and once for the user (during home-manager switch).

Mitigation: By keeping them separate, you accept this double evaluation cost in exchange for the flexibility to update them independently. This is a valid trade-off. Optimization: We can't speed up the evaluation itself without

merging flakes, but we **can** speed up your workflow by making the code easier to edit. You won't have to edit 3 files to add one package.

1.3 3. Implementation Steps

1.3.1 Step 1: Create the Module Structure

We will move the existing configuration from `hosts/*/home-manager/*` into the new `modules/` directory.

- `home.sessionVariables` and `home.username` -> `modules/core/default.nix`
- `programs.fish` -> `modules/shell/fish.nix`
- `wayland.windowManager.hyprland` -> `modules/desktop/hyprland.nix`

1.3.2 Step 2: Group Packages

The monolithic `packages/apps.nix` will be split.

- `modules/packages/base.nix`: Firefox/Zen, MPV, Nautilus, LibreOffice.
- `modules/packages/dev.nix`: Git, Helix/Neovim, Compilers.
- `modules/packages/creative.nix`: Krita, Inkscape, OBS.

1.3.3 Step 3: Define Hosts

The `hosts/t490s.nix` file becomes a simple list of imports:

```
{ pkgs, ... }: {
  imports = [
    ../modules/core
    ../modules/shell/fish.nix
    ../modules/desktop/hyprland.nix
    ../modules/packages/base.nix
    ../modules/packages/dev.nix
    ../modules/packages/creative.nix
  ];

  # Host-specific overrides (e.g. monitor config)
  wayland.windowManager.hyprland.settings.monitor = [
    "eDP-1, 1920x1080, 0x0, 1"
  ];
}
```

This structure allows you to add a new server or laptop in seconds by just picking the modules it needs.